Sistema de Gestión Documental

Manual de usuario

1. Objetivo

Utilizar una aplicación web para organizar, consultar y administrar documentos digitales con acceso controlado.

2. Iniciar

- Confirmar que el servidor y MongoDB estén activos.
- Ejecutar en backend con cd servidor y npm run dev
- Ejecutar el frontend con cd cliente y npm start
- Abrir http://localhost:4200 en el navegador.
- Introducir el correo y la contraseña de una cuenta registrada y seleccionar Iniciar sesión.

3. Archivos

- Buscar: escribir parte del nombre y ejecutar la búsqueda.
- Subir: seleccionar un archivo y, si corresponde, una carpeta de destino.
- Descargar: usar Descargar en el archivo deseado.
- Renombrar: escribir el nuevo nombre y confirmar.
- Mover: elegir una carpeta de destino.
- Papelera: enviar el documento a la papelera cuando ya no se necesite.

4. Carpetas

- Abrir Carpetas, escribir un nombre y elegir una carpeta padre para crear una subcarpeta; dejar la selección raíz para crear una carpeta principal. Seleccionar Crear carpeta.
- Mover carpera: elegir la carpeta padre y mover
- Eliminar carpeta: mandar a papelera
- Eliminar definitivamente: desde papelera

5. Papelera

Abrir Papelera para consultar los recursos eliminados. Restaurar recupera un recurso si el servidor lo permite. Eliminar definitivamente es irreversible; confirmar con cuidado. La eliminación de carpetas puede afectar a su contenido.

6. **Permisos, almacenamiento y reportes**

Los niveles de acceso previstos son LECTURA, COMENTAR y EDICION. El sistema también contempla almacenamiento, historial, administración de usuarios y reportes.

7. **Problemas frecuentes**

- No inicia sesión: comprobar credenciales y conexión.
- No aparecen documentos: verificar la cuenta y actualizar la lista.
- No se sube un archivo: revisar cuota, tamaño y estado del servidor.
- Error de acceso: iniciar sesión de nuevo y comprobar permisos.
- Página no disponible: confirmar que MongoDB, API y cliente estén en ejecución.
- No compartir contraseñas y evitar eliminar definitivamente documentos importantes.

Manual Técnico

1. **Arquitectura**

- Cliente independiente: Angular 21.
- API REST: Node.js
- Persistencia de metadatos: MongoDB.
- Contenido binario: GridFS.

El cliente no debe conectarse directamente a MongoDB.

2. **Inicio local**

- Instalar Node.js/npm y MongoDB.
- En servidor/, ejecutar npm install.
- Configurar las variables de entorno que espera el servidor, sin subir secretos al repositorio.
- Iniciar el servidor con el script definido en servidor/package.json (durante el desarrollo se ha utilizado npm run dev).
- En cliente/, ejecutar npm install y npm start.
- Abrir http://localhost:4200.

5. Seguridad y consistencia

El servidor debe validar autenticación, propiedad y permisos en cada operación; no basta con ocultar botones. La jerarquía de carpetas no debe permitir ciclos. Los

recursos en papelera deben excluirse de listados normales y la actividad debe conservarse para auditoría.

6. Lista de comprobación

Ejecutar npm run build en cliente. Probar login, token inválido, carga, búsqueda, descarga, renombrado, movimiento, papelera, restauración, borrado definitivo, permisos entre usuarios, rol administrador y reportes. Verificar además la carga inicial de datos.

Documentación del modelo documental

Este documento describe entidades lógicas derivadas de los módulos y requisitos conocidos. Contrastar los nombres y campos exactos con los esquemas Mongoose.

1. **Usuario**

Cuenta del sistema: nombre, correo, contraseña, rol, estado y datos de almacenamiento. El correo debería tener índice único.

2. **Carpeta**

Carpeta propiedad de un usuario, con nombre y referencia opcional a una carpeta padre. La referencia padre permite representar una jerarquía. El servidor debe impedir ciclos y referencias inválidas.

3. **Archivo**

Metadatos de un documento: identificador, nombre, tipo/extensión, tamaño, propietario, carpeta opcional, fecha y referencia al identificador del objeto binario de GridFS.

4. **GridFS**

MongoDB GridFS almacena contenido binario en colecciones de archivos y fragmentos (habitualmente fs.files y fs.chunks, salvo configuración distinta). Los metadatos del archivo deben enlazarse al identificador GridFS; una ruta local no sustituye al contenido.

5. **Permiso**

Relación entre un recurso compartido y un usuario receptor, con tipo de recurso y nivel como LECTURA o EDICION. El servidor debe validar la autorización en cada operación.

6. **Actividad**

Registro de auditoría que identifica usuario, acción, recurso y fecha/hora.

7. **Relaciones lógicas**

- Un usuario puede poseer múltiples carpetas y archivos.
- Una carpeta puede contener archivos y subcarpetas.
- Un archivo referencia su contenido binario en GridFS.
- Un recurso puede tener varios permisos.
- Un usuario puede producir múltiples registros de actividad.
- Usuario: correo normalizado único.
- Archivo: propietario/estado y carpeta/nombre.
- Carpeta: propietario/carpeta padre.
- Permiso: recurso/usuario receptor.
- Actividad: usuario/fecha.
- Reglas de consistencia
- No exceder cuota; evitar ciclos; validar acceso a recursos; mantener ubicación al mover; excluir papelera de listados normales; restaurar o borrar sin referencias inconsistentes.